

# TP 4: Classification supervisée

September 14, 2023

## 0.1 Introduction

Dans ce TP, nous allons explorer plusieurs algorithmes de classification en utilisant la bibliothèque `scikit-learn` dans Python. La classification supervisée est une tâche cruciale en data mining, où nous formons des modèles à partir de données étiquetées pour prédire correctement les étiquettes de nouvelles données.

### 0.1.1 Objectif

L'objectif principal de ce TP est de vous familiariser avec divers algorithmes de classification et de comprendre comment les appliquer à des ensembles de données réels. Pour cela, nous allons utiliser les algorithmes suivants :

- k-Nearest Neighbors (k-NN) avec `KNeighborsClassifier`
- Arbre de décision avec `DecisionTreeClassifier`
- Forêt aléatoire avec `RandomForestClassifier`
- Naïve Bayes Gaussien avec `GaussianNB`
- Support Vector Machine (SVM) avec `SVC`
- Naïve Bayes Bernoulli avec `BernoulliNB`
- Régression logistique avec `LogisticRegression`
- Stochastic Gradient Descent (SGD) avec `SGDClassifier`
- AdaBoost avec `AdaBoostClassifier`
- Gradient Boosting avec `GradientBoostingClassifier`
- XGBoost avec `XGBClassifier`

```
[ ]: import pandas as pd
from sklearn.model_selection import train_test_split
import warnings

warnings.filterwarnings("ignore")
```

```
[14]: from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

from sklearn.naive_bayes import BernoulliNB
from sklearn.linear_model import LogisticRegression, SGDClassifier
```

```
from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier
```

## 0.2 préparation des données

Le code suivant effectue des tâches de préparation des données qui ont été précédemment abordées lors des TPs précédentes. Ces tâches comprennent l'importation des données, le filtrage des enregistrements, la suppression des valeurs manquantes, la normalisation des données, l'encodage des variables catégorielles, et la division du jeu de données en sous-ensembles d'entraînement et de test. Cette préparation des données est réalisée sur l'ensemble de données du Titanic, et le jeu de données préparé résultant sera utilisé ultérieurement pour l'entraînement de divers modèles d'apprentissage automatique.

```
[78]: df = pd.read_excel('Datasets/titanic3.xls')
df = df.drop(['name', 'sibsp', 'parch', 'ticket', 'cabin', 'embarked', 'boat', 'body', 'home.dest'], axis=1)

numerical_features = ['pclass', 'age', 'fare']
categorical_features = ['sex']
df.head()
```

```
[78]:
```

|   | pclass | survived | sex    | age     | fare     |
|---|--------|----------|--------|---------|----------|
| 0 | 1      | 1        | female | 29.0000 | 211.3375 |
| 1 | 1      | 1        | male   | 0.9167  | 151.5500 |
| 2 | 1      | 0        | female | 2.0000  | 151.5500 |
| 3 | 1      | 0        | male   | 30.0000 | 151.5500 |
| 4 | 1      | 0        | female | 25.0000 | 151.5500 |

```
[4]: from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
```

```
[81]: imputer_num = SimpleImputer(strategy='mean')
imputer_cat = SimpleImputer(strategy='most_frequent')
Scaler = StandardScaler()
encoder = OneHotEncoder()
df[['pclass', 'age', 'fare']] = imputer_num.fit_transform(df[['pclass', 'age', 'fare']])
df[['pclass', 'age', 'fare']] = Scaler.fit_transform(df[['pclass', 'age', 'fare']])

df[['sex']] = imputer_cat.fit_transform(df[['sex']])
def Conv_sex_to_number(sex):
    if sex == 'female':
        return 0
    else:
        return 1
df['sex'] = df['sex'].map(Conv_sex_to_number)
```

```
df.head()
```

```
[81]:      pclass  survived  sex      age      fare
0 -1.546098         1    0 -0.068420  3.442480
1 -1.546098         1    1 -2.249092  2.286476
2 -1.546098         0    0 -2.164974  2.286476
3 -1.546098         0    1  0.009230  2.286476
4 -1.546098         0    0 -0.379021  2.286476
```

```
[82]: y = df['survived']
X = df.drop('survived', axis=1)

X_train, X_test, y_train, y_test = train_test_split(X, y)
X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

```
[82]: ((981, 4), (328, 4), (981,), (328,))
```

## 1 Processus d'entraînement

Schéma du processus d'entraînement d'un modèle dans Scikit-Learn :

1. **Importez le modèle** : Tout d'abord, importez le modèle que vous souhaitez utiliser. Par exemple, pour un modèle de régression linéaire :

```
from sklearn.linear_model import LinearRegression
```

2. **Initialisez le modèle** : Créez une instance du modèle en spécifiant les hyperparamètres souhaités. Par exemple :

```
model = LinearRegression()
```

Vous pouvez également spécifier des hyperparamètres spécifiques, par exemple :

```
model = LinearRegression(fit_intercept=True, normalize=False)
```

3. **Entraînez le modèle** : Utilisez la méthode `fit` pour entraîner le modèle sur vos données d'entraînement (`X`, `y`) :

```
model.fit(X_train, y_train)
```

Le modèle apprend à partir des données d'entraînement et ajuste ses paramètres pour minimiser l'erreur.

4. **Évaluez le modèle** : Vous pouvez évaluer la performance du modèle en utilisant des méthodes telles que `score`. Par exemple, pour un modèle de régression linéaire, vous pouvez utiliser la méthode `score` pour calculer le coefficient de détermination ( $R^2$ ) :

```
score = model.score(X_test, y_test)
```

En fonction du type de modèle que vous utilisez, il existe d'autres méthodes spécifiques pour évaluer la performance, telles que `predict` pour effectuer des prédictions ou `predict_proba` pour obtenir des probabilités dans le cas de la classification.

5. **Utilisez le modèle entraîné** : Une fois que le modèle est entraîné et évalué, vous pouvez l'utiliser pour faire des prédictions sur de nouvelles données non vues :

```
y_pred = model.predict(X_new)
```

Vous pouvez également accéder à d'autres attributs et méthodes spécifiques au modèle, en fonction du type de modèle que vous utilisez. Par exemple, pour un modèle de régression linéaire, vous pouvez accéder aux coefficients :

```
coefficients = model.coef_
```

Chaque type de modèle peut fournir des méthodes et des attributs spécifiques en fonction de son utilisation.

## 2 K-Nearest Neighbors (KNN)

Le modèle KNN est basé sur la distance entre les points de données. Voici quelques hyperparamètres importants :

- `n_neighbors` : Le nombre de voisins à considérer.
- `weights` : La méthode de pondération des voisins (uniforme ou inverse de la distance).
- `metric` : La mesure de distance utilisée (par exemple, la distance Euclidienne).

```
[83]: model = ↪ KNeighborsClassifier(n_neighbors=3, weights='distance', metric='minkowski')
model.fit(X_train, y_train)
model.score(X_test, y_test)
```

```
[83]: 0.774390243902439
```

## 3 Arbres de Décision

Les arbres de décision sont des modèles basés sur des règles de décision. Voici quelques hyperparamètres importants :

- `criterion` : Le critère de division de l'arbre (par exemple, "gini" ou "entropy").
- `max_depth` : La profondeur maximale de l'arbre.
- `min_samples_split` : Le nombre minimum d'échantillons requis pour diviser un nœud.

```
[147]: model = ↪ DecisionTreeClassifier(criterion='entropy', max_depth=3, min_samples_split=20)
model.fit(X_train, y_train)
model.score(X_test, y_test)
```

```
[147]: 0.801829268292683
```

```
[149]: from sklearn.tree import export_graphviz
import graphviz

# Visualize the decision tree
```

```
dot_data = export_graphviz(model, out_file=None,
                           feature_names=X_train.columns[:],
                           class_names=['survived', 'not survived'],
                           filled=True, rounded=True,
                           special_characters=True)

graph = graphviz.Source(dot_data)
graph.format = 'png'
graph.render("iris_decision_tree")
```

[149]: 'iris\_decision\_tree.png'

## 4 Random Forest

Random Forest est un modèle basé sur l'ensemble d'arbres de décision. En plus des hyperparamètres des arbres de décision, il y a des hyperparamètres spécifiques :

- `n_estimators` : Le nombre d'arbres dans la forêt.
- `max_features` : Le nombre maximal de caractéristiques à considérer pour chaque division.
- `bootstrap` : Indique si les échantillons doivent être tirés avec remplacement.

```
[153]: model = RandomForestClassifier(n_estimators=6, criterion='entropy', max_depth=2, min_samples_split=10)
model.fit(X_train, y_train)
model.score(X_test, y_test)
```

[153]: 0.7591463414634146

```
[157]: import pydotplus
import matplotlib.pyplot as plt
import itertools

# Create a grid for subplots (3 rows, 2 columns)
fig, axes = plt.subplots(3, 2, figsize=(12, 10))

for i, ax in enumerate(itertools.chain(*axes)):
    individual_tree = model.estimators_[i]
    dot_data = export_graphviz(individual_tree, out_file=None,
                              feature_names=X_train.columns[:],
                              class_names=['survived', 'not survived'],
                              filled=True, rounded=True,
                              special_characters=True)

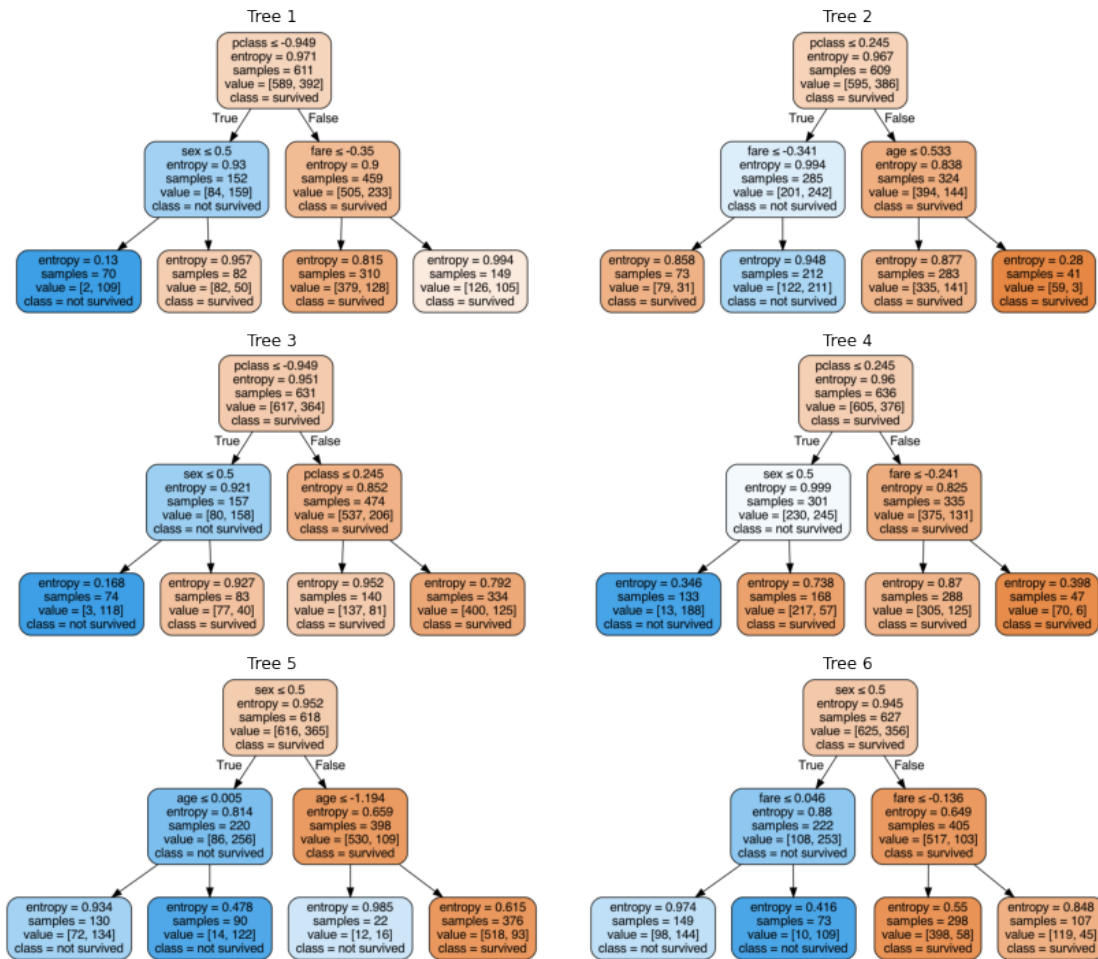
    graph = graphviz.Source(dot_data) # Use graphviz.Source to render the graph
    graph.format = 'png'
    graph.render(f'tree_{i+1}') # Save the graph as a PDF (you can change the
    # format if needed)
```

```

img = plt.imread(f'tree_{i+1}.png') # Read the saved PDF as an image
ax.imshow(img)
ax.set_title(f'Tree {i+1}')
ax.axis('off')

plt.tight_layout()
plt.show()

```



## 4.1 Naive Bayes

Les modèles Naive Bayes sont basés sur le théorème de Bayes. Voici un hyperparamètre courant :

- *alpha* : Le paramètre de lissage Laplace pour gérer les probabilités nulles.

```

[86]: model = GaussianNB()
model.fit(X_train, y_train)
model.score(X_test, y_test)

```

[86]: 0.7530487804878049

## 4.2 Support Vector Machine (SVM)

Les SVM sont basés sur la recherche du meilleur hyperplan de séparation. Voici quelques hyperparamètres importants :

- **C** : Le paramètre de régularisation.
- **kernel** : La fonction de noyau utilisée (par exemple, linéaire, RBF, polynomial).

```
[87]: model = SVC(C=1, kernel='poly', degree=3, gamma='auto', coef0=0, shrinking=True, probability=False)
      model.fit(X_train, y_train)
      model.score(X_test, y_test)
```

[87]: 0.7804878048780488

### 4.2.1 2. Régression Logistique (LogisticRegression)

**Description du Modèle** : La Régression Logistique est un modèle de classification qui est largement utilisé pour résoudre des problèmes de classification binaire. Il est basé sur une fonction logistique qui estime la probabilité qu'une instance appartienne à une classe donnée.

**Processus de Fonctionnement** : Le modèle de Régression Logistique calcule la probabilité qu'une instance appartienne à une classe en utilisant la fonction logistique. Il ajuste les coefficients en minimisant la fonction de coût, généralement la perte de log-vraisemblance.

**Hyperparamètres Importants** : - **C** : Inverse de la force de régularisation (plus la valeur est petite, plus la régularisation est forte). - **penalty** : Type de régularisation ('l1', 'l2', 'elasticnet', 'none'). - **solver** : Algorithme d'optimisation ('lbfgs', 'liblinear', 'newton-cg', 'sag', 'saga').

```
[89]: model = LogisticRegression(C=1, kernel='poly', degree=3, gamma='auto', coef0=0, shrinking=True, probability=False)
      model.fit(X_train, y_train)
      model.score(X_test, y_test)
```

[89]: 0.7804878048780488

### 4.2.2 3. SGD Classifier (SGDClassifier)

**Description du Modèle** : Le modèle SGD Classifier est un classificateur linéaire basé sur la descente de gradient stochastique (SGD). Il est efficace pour la classification linéaire et prend en charge la régularisation.

**Processus de Fonctionnement** : Le modèle SGD Classifier optimise une fonction de perte en utilisant la descente de gradient stochastique. Il peut être utilisé pour une variété de tâches de classification linéaire.

**Hyperparamètres Importants** : - **alpha** : Paramètre de régularisation. - **loss** : Fonction de perte ('hinge' pour SVM linéaire, 'log' pour régression logistique, etc.). - **penalty** : Type de

régularisation ('l1', 'l2', 'elasticnet').

```
[90]: model = SVC(C=1, kernel='poly', degree=3, gamma='auto', coef0=0, shrinking=True, probability=False)
      model.fit(X_train, y_train)
      model.score(X_test, y_test)
```

[90]: 0.7804878048780488

#### 4.2.3 4. AdaBoost Classifier (AdaBoostClassifier)

**Description du Modèle :** Le modèle AdaBoost Classifier est un algorithme d'ensemble qui améliore les performances des classificateurs faibles en les combinant. Il attribue des poids aux échantillons pour mettre l'accent sur les exemples mal classés.

**Processus de Fonctionnement :** AdaBoost ajuste une série de classificateurs faibles de manière itérative. Il attribue des poids différents aux échantillons pour corriger les erreurs des classificateurs précédents.

**Hyperparamètres Importants :** - `n_estimators` : Nombre de classificateurs faibles à utiliser. - `learning_rate` : Taux d'apprentissage pour contrôler la contribution de chaque classificateur faible. - `base_estimator` : Le classificateur faible à utiliser (par défaut, c'est un arbre de décision peu profond).

```
[91]: model = make_pipeline(preprocessor, AdaBoostClassifier())
      model.fit(X_train, y_train)
      model.score(X_test, y_test)
```

[91]: 0.7896341463414634

#### 4.2.4 5. Gradient Boosting Classifier (GradientBoostingClassifier)

**Description du Modèle :** Le modèle Gradient Boosting Classifier est un algorithme d'ensemble qui construit des arbres de décision de manière itérative. Il est très efficace et est utilisé pour des tâches de classification et de régression.

**Processus de Fonctionnement :** Gradient Boosting construit des arbres de décision peu profonds de manière itérative pour corriger les erreurs des arbres précédents. Il utilise la descente de gradient pour minimiser la fonction de perte.

**Hyperparamètres Importants :** - `n_estimators` : Nombre d'arbres de décision à construire. - `learning_rate` : Taux d'apprentissage pour contrôler la contribution de chaque arbre. - `max_depth` : Profondeur maximale des arbres de décision.

```
[92]: model = GradientBoostingClassifier()
      model.fit(X_train, y_train)
      model.score(X_test, y_test)
```

[92]: 0.7804878048780488



#### 4.2.5 6. XGBoost Classifier (XGBClassifier)

**Description du Modèle :** Le modèle XGBoost Classifier est une implémentation efficace du Gradient Boosting qui est devenu très populaire dans les compétitions de science des données. Il est très

performant et offre une grande flexibilité.

**Processus de Fonctionnement :** XGBoost construit des arbres de décision de manière itérative, en utilisant des techniques d'optimisation avancées. Il prend en charge la régularisation, la gestion des valeurs manquantes et est hautement parallélisable.

**Hyperparamètres Importants :** - `n_estimators` : Nombre d'arbres de décision à construire. - `learning_rate` : Taux d'apprentissage pour contrôler la contribution de chaque arbre. - `max_depth` : Profondeur maximale des arbres de décision.

Chacun de ces modèles a ses avantages et inconvénients, ainsi que des hyperparamètres spécifiques qui peuvent être ajustés pour optimiser les performances en fonction du problème de classification ou de régression que vous essayez de résoudre.

```
[93]: model = XGBClassifier()
      model.fit(X_train, y_train)
      model.score(X_test, y_test)
```

```
[93]: 0.8109756097560976
```

## 5 Comparaison des Modèles de Machine Learning : Quel Algorithme Choisir ?

Choisir le bon modèle pour une tâche d'apprentissage automatique dépend de plusieurs facteurs, notamment le type de données que vous avez, la nature de la tâche (classification ou régression), la taille de l'ensemble de données et d'autres considérations spécifiques au problème. Voici un guide général pour choisir parmi les modèles que vous avez mentionnés :

### 1. K-Nearest Neighbors (KNN) Classifier :

- Type de données : Convient aux données de toutes sortes, en particulier lorsque la structure de voisinage est importante.
- Situation : Utilisé dans des cas où il est difficile de modéliser la distribution sous-jacente des données.

### 2. Arbre de décision (Decision Tree) et Random Forest :

- Type de données : Adapté aux données tabulaires ou structurées.
- Situation : Bon pour la classification binaire ou multiclasse, la régression et l'interprétabilité des modèles.

### 3. Naive Bayes (GaussianNB et BernoulliNB) :

- Type de données : Convient aux données textuelles, catégorielles et à certaines données numériques.
- Situation : Souvent utilisé pour la classification de textes, la détection de spam et d'autres tâches de classification de documents.

### 4. Support Vector Machine (SVM) :

- Type de données : Convient aux données tabulaires et numériques.

- Situation : Efficace pour la classification binaire et la régression, en particulier lorsque les données sont linéairement séparables ou peuvent être transformées pour l'être.
5. **Régression logistique (Logistic Regression)** :
    - Type de données : Convient aux données tabulaires et numériques.
    - Situation : Souvent utilisée pour la classification binaire et multiclasse, en particulier lorsque l'interprétabilité du modèle est importante.
  6. **Gradient Boosting (GradientBoostingClassifier)** :
    - Type de données : Convient aux données tabulaires ou structurées.
    - Situation : Performe généralement bien dans un large éventail de tâches de classification et de régression, en particulier lorsque l'accent est mis sur la précision.
  7. **AdaBoost (AdaBoostClassifier)** :
    - Type de données : Convient aux données tabulaires ou structurées.
    - Situation : Efficace pour la classification binaire, en particulier lorsque vous avez un ensemble de données déséquilibré.
  8. **XGBoost (XGBClassifier)** :
    - Type de données : Convient aux données tabulaires ou structurées.
    - Situation : Performe bien dans de nombreuses tâches, en particulier lorsque la vitesse d'exécution et la précision sont essentielles.

La clé pour choisir le bon modèle est de comprendre les caractéristiques de vos données et les exigences de votre tâche. Il est souvent recommandé d'expérimenter avec plusieurs modèles et de choisir celui qui donne les meilleurs résultats lors de la validation croisée. La complexité du modèle, la taille de l'ensemble de données et d'autres facteurs peuvent également influencer votre choix.

## 6 Exercice :

### 6.1 Exercice 1:

Dans le contexte du code fourni, qui visualise les frontières de décision pour différents modèles, votre tâche consiste à créer des visualisations de ces frontières de décision en faisant varier systématiquement les hyperparamètres. Votre objectif est d'expliquer et d'illustrer visuellement l'impact direct de ces hyperparamètres sur les frontières de décision.

```
[41]: from sklearn.metrics import accuracy_score, confusion_matrix, roc_auc_score,
      ↪ ConfusionMatrixDisplay, precision_score, recall_score, f1_score,
      ↪ classification_report, roc_curve, plot_roc_curve, auc,
      ↪ precision_recall_curve, plot_precision_recall_curve, average_precision_score
      from sklearn.model_selection import cross_val_score
```

```
[178]: from tqdm import tqdm
      models = []

      models.append(['KNeighbors', KNeighborsClassifier()])
      models.append(['KNeighbors k = 40', KNeighborsClassifier(n_neighbors=40)])
      models.append(['KNeighbors k = 1', KNeighborsClassifier(n_neighbors=1)])
      models.append(['GaussianNB', GaussianNB()])
      models.append(['Decision Tree', DecisionTreeClassifier()])
      models.append(['Random Forest', RandomForestClassifier()])
```

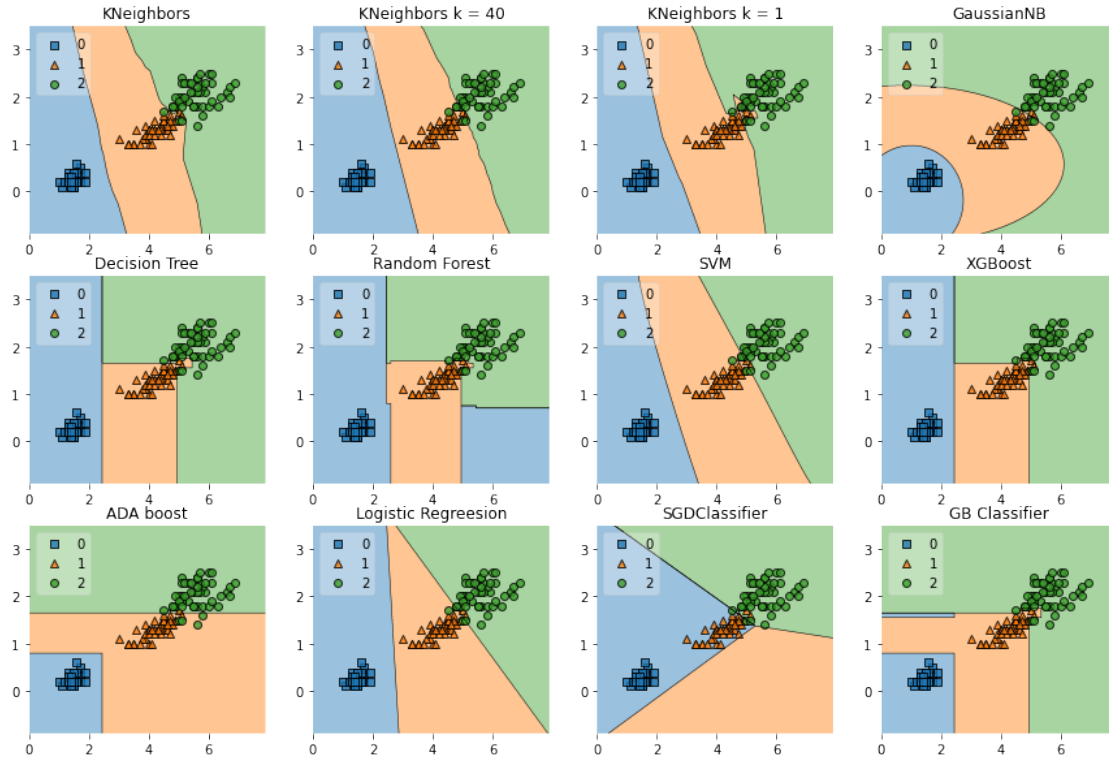
```
models.append(['SVM', SVC(probability=True)])
models.append(['XGBoost', XGBClassifier()])
models.append(['ADA boost', AdaBoostClassifier()])
models.append(['Logistic Regreesion', LogisticRegression()])
models.append(['SGDClassifier', SGDClassifier(loss='log')])
models.append(['GB Classifier', GradientBoostingClassifier()])
```

```
[160]: from sklearn import datasets
iris = datasets.load_iris()
X = iris.data[:, [2, 3]]
y = iris.target
```

```
[171]: from mlxtend.plotting import plot_decision_regions
plt.subplots(3,4,figsize=(15,10))
for idx, model in enumerate(models,start=1):

    print(model[0])
    model[1].fit(X,y) # Plotting decision region
    plt.subplot(3,4,idx)
    plot_decision_regions(X,y, clf=model[1], legend=2) # Adding axes
    ↪ annotations
    score =model[1].score(X,y) *100
    plt.title(f'{model[0]} {score:0.2f}')
```

KNeighbors  
KNeighbors k = 40  
KNeighbors k = 1  
GaussianNB  
Decision Tree  
Random Forest  
SVM  
XGBoost  
ADA boost  
Logistic Regreesion  
SGDClassifier  
GB Classifier



## 6.2 Exercice 2:

Puisqu'il n'y a pas de solution unique pour déterminer le meilleur modèle, nous souhaitons entraîner tous les modèles sur nos données et choisir le modèle le plus performant. Complétez donc le code suivant pour illustrer les performances des modèles en considérant les métriques suivantes : 'Précision', 'Recall', 'F1', 'AUC-ROC', 'Accuracy', 'Moyenne de la Précision en K-Fold', 'Écart-type'.

```
[ ]: lst_1 = [] # Une liste vide pour stocker les résultats finaux

# Boucle sur chaque modèle dans la liste "models"
for m in tqdm(range(len(models))): # Utilisation de "tqdm" pour afficher une
    ↪ barre de progression
    lst_2 = [] # Une liste vide pour stocker les résultats de chaque modèle

    # Obtient le modèle actuel à partir de la liste "models"
    model = models[m][1]
    # Entraîne le modèle sur les données d'entraînement
    model.fit(X_train, y_train)

    # Prédit les étiquettes des données de test
    y_pred = #TODO
```

```

# Prédit les scores de probabilité de classe 0 (ou classe positive)
y_score = model.predict_proba(X_test)[: , 0]

# Calcule la matrice de confusion pour évaluer les performances du modèle
cm = #TODO

# Effectue une validation croisée en utilisant 10 plis pour obtenir des
↳ précisions
accuracies = #TODO

# Calcule le score ROC AUC pour évaluer la capacité de discrimination du
↳ modèle
roc = #TODO

# Calcule la précision du modèle
precision = #TODO

# Calcule le rappel du modèle
recall = #TODO

# Calcule le score F1 du modèle
f1 = #TODO

# Ajoute les résultats dans la liste "lst_2"
lst_2.append(models[m][0]) # Nom du modèle
lst_2.append((accuracy_score(y_test, y_pred)) * 100) # Score d'exactitude
↳ en pourcentage
lst_2.append(accuracies.mean() * 100) # Précision moyenne en pourcentage
↳ lors de la validation croisée
lst_2.append(accuracies.std() * 100) # Écart-type de la précision lors de
↳ la validation croisée
lst_2.append(roc) # Score ROC AUC
lst_2.append(precision) # Score de précision
lst_2.append(recall) # Score de rappel
lst_2.append(f1) # Score F1

# Ajoute les résultats du modèle actuel dans la liste principale "lst_1"
lst_1.append(lst_2)

# À ce stade, "lst_1" contient les résultats pour chaque modèle

```

```

[124]: df = pd.DataFrame(lst_1, columns= ['Model', 'Accuracy', 'K-Fold Mean Accuracy',
↳ 'Std. Deviation', 'ROC AUC', 'Precision', 'Recall', 'F1'])
df.sort_values(by= ['Accuracy', 'K-Fold Mean Accuracy'], inplace= True,
↳ ascending= False)
df

```

[124] :

|    | Model               | Accuracy  | K-Fold Mean Accuracy | Std. Deviation | \ |
|----|---------------------|-----------|----------------------|----------------|---|
| 6  | XGBoost             | 81.402439 | 79.004329            | 3.239766       |   |
| 0  | KNeighbors          | 80.487805 | 78.492063            | 2.710225       |   |
| 3  | Random Forest       | 80.182927 | 78.188002            | 1.743444       |   |
| 7  | ADA boost           | 78.963415 | 78.078747            | 2.851160       |   |
| 2  | Decision Tree       | 78.658537 | 75.132962            | 2.796155       |   |
| 4  | SVM                 | 78.353659 | 78.079777            | 3.732553       |   |
| 10 | GB Classifier       | 78.048780 | 81.247166            | 3.041509       |   |
| 8  | Logistic Regreesion | 75.914634 | 78.690992            | 3.263168       |   |
| 1  | GaussianNB          | 75.000000 | 77.264482            | 2.528468       |   |
| 5  | BernoulliNB         | 74.390244 | 77.468563            | 2.460074       |   |
| 9  | SGDClassifier       | 74.085366 | 77.370645            | 4.296353       |   |

|    | ROC AUC  | Precision | Recall   | F1       |
|----|----------|-----------|----------|----------|
| 6  | 0.132925 | 0.789916  | 0.723077 | 0.755020 |
| 0  | 0.157401 | 0.784483  | 0.700000 | 0.739837 |
| 3  | 0.148310 | 0.768595  | 0.715385 | 0.741036 |
| 7  | 0.150602 | 0.756303  | 0.692308 | 0.722892 |
| 2  | 0.219406 | 0.727273  | 0.738462 | 0.732824 |
| 4  | 0.185334 | 0.839080  | 0.561538 | 0.672811 |
| 10 | 0.124262 | 0.795918  | 0.600000 | 0.684211 |
| 8  | 0.170688 | 0.721739  | 0.638462 | 0.677551 |
| 1  | 0.197028 | 0.690476  | 0.669231 | 0.679687 |
| 5  | 0.193201 | 0.739583  | 0.546154 | 0.628319 |
| 9  | 0.190579 | 0.643312  | 0.776923 | 0.703833 |

Parmi les modèles testés précédemment, identifiez les trois modèles les plus performants. Ensuite, effectuez un ajustement des hyperparamètres pour obtenir le meilleur modèle possible.

### 6.3 Exercice 3:

Poursuivez le travail effectué lors du TP précédent sur l'ensemble de données sur les AVC. L'objectif est de trouver le meilleur modèle pour prédire les AVC.